

torch.compiler.allow_in_graph#

https://pytorch.org/docs/stable/generated/torch.compiler.allow_in_graph.html

Description:

Tells the compiler frontend (Dynamo) to skip symbolic introspection of the function and instead directly write it to the graph when encountered. If you are using `torch.compile()` (with `backend="inductor"` (the default)), or `torch.export.export()`, and trying to black-box a Python function throughout all tracing, do not use this API. Instead, please create a custom operator (see PyTorch Custom Operators Landing Page) If you're a typical `torch.compile` user (e.g. you're applying `torch.compile` to a model to make it run faster), you probably don't want to use this function. `allow_in_graph()` is a footgun because it skips the compiler frontend (Dynamo) that is responsible for doing safety checks (graph breaks, handling closures, etc). Incorrect usage will lead to difficult-to-debug silent incorrectness issues. Given a Python function with no `allow_in_graph` decorator, regular execution of `torch.compile` traces through the function. `allow_in_graph()` changes it so that the frontend does not trace inside the function, but the compiler backend still traces through it. Compare this to custom operators, which treats a function as a black box throughout the `torch.compile` stack. The following table

Function Signature

```
torch.compiler.allow_in_graph(fn)[source]#
```

Parameters

Code Examples

```
torch.compiler.allow_in_graph(my_custom_function)
```

```
@torch.compile(...)
```

```
def fn(x):
```

```
    x = torch.add(x, 1)
```

```
    x = my_custom_function(x)
```

```
    x = torch.add(x, 1)
```

```
    return x
```

```
fn(...)
```

Notes & Warnings

WARNING

If you're a typical `torch.compile` user (e.g. you're applying `torch.compile` to a model to make it run faster), you probably don't want to use this function. `allow_in_graph()` is a footgun because it skips the compiler frontend (Dynamo) that is responsible for doing safety checks (graph breaks, handling closures, etc). Incorrect usage will lead to difficult-to-debug silent incorrectness issues.

Detailed Documentation

Documentation

Tells the compiler frontend (Dynamo) to skip symbolic introspection of the function and instead directly write it to the graph when encountered.

If you are using `torch.compile()` (with `backend="inductor"` (the default)), or `torch.export.export()`, and trying to black-box a Python function throughout all tracing, do not use this API. Instead, please create a custom operator (see [PyTorch Custom Operators Landing Page](#))

If you're a typical `torch.compile` user (e.g. you're applying `torch.compile` to a model to make it run faster), you probably don't want to use this function. `allow_in_graph()` is a footgun because it skips the compiler frontend (Dynamo) that is responsible for doing safety checks (graph breaks, handling closures, etc). Incorrect usage will lead to difficult-to-debug silent incorrectness issues.

Given a Python function with no `allow_in_graph` decorator, regular execution of `torch.compile` traces through the function. `allow_in_graph()` changes it so that the frontend does not trace inside the function, but the compiler backend still traces through it. Compare this to custom operators, which treats a function as a black box throughout the `torch.compile` stack. The following table compares these mechanisms.

Backend (AOTAutograd+Inductor)

One common use case for `allow_in_graph()` is as an escape hatch for the compiler frontend: if you know the function works w.r.t. to the downstream components of the compilation stack (AOTAutograd and Inductor) but there is a Dynamo bug that prevents it from symbolically introspecting the function properly (or if your code is in C/C++ and therefore cannot be introspected with Dynamo), then one can decorate said function with `allow_in_graph()` to bypass Dynamo.

We require that `fn` adhere to the following restrictions. Failure to adhere results in undefined behavior:

The inputs to `fn` must be Proxy-able types in the FX graph. Valid types include:
Tensor/int/bool/float/None/List[Tensor?]/List[int?]/List[float?] Tuple[Tensor?, ...]/Tuple[int?, ...]/Tuple[float?, ...]/torch.dtype/torch.device

The outputs to `fn` must be Proxy-able types in the FX graph (see previous bullet)

all Tensors used inside of `fn` must be passed directly as inputs to `fn` (as opposed to being captured variables).

`fn` – A callable representing the function to be included in the graph. If `fn` is a list or tuple of callables it recursively applies `allow_in_graph()` to each function and returns a new list or tuple containing the modified functions.

Will capture a single graph containing `my_custom_function()`.

